

3D-SCPTOP: model, reformulation and solution method

Working document – formulation, algorithms and computational study

August 30, 2026

Contents

1	Mathematical Model	1
1.1	Symmetry breaking and slot-set sizing	6
2	Branch and Price Solution Approach	6
2.1	Formulation of the Master Problem and the Restricted Master Problem	7
2.2	Lower bounds	9
2.3	Pricing Subproblem	9
2.4	Initial Columns	12
2.5	Branching	12
2.6	Two corrections to the compact model	12
3	Computational Experiments	13
3.1	Instance generation	14
3.2	Methods compared	14
3.3	Results	15
3.4	Discussion	18

1 Mathematical Model

The 3D-SCPTOP formulation integrates stack-based truck-loading constraints with procurement transport and inventory decisions. It departs from a standard 3D container-loading formulation in two related ways. First, the 3D placement of individual RTIs inside the truck is replaced by a 2D placement of variable-composition *stacks* on the truck floor, with per-stack height and weight caps. Second, the per-period decision of which product fills each slot is retained, but the crush-limit cap is evaluated slot-by-slot using the period’s actual product mix. This yields a tighter per-stack weight bound than a worst-case-per-RTI-type abstraction, while eliminating vertical-stability constraints altogether, since stacks are self-supporting by construction.

Notational conventions. Throughout this section, $g \in G = \{s, p\}$ indexes nodes (supplier and plant), $n \in N$ products, $r \in R$ RTI types, $h \in H$ stack types, $i, j \in I$ stack-slot positions within a loading configuration, $l \in L$ loading configurations, $k \in K$ trucks and $t \in T$ planning periods. Variables whose direction is determined by definition omit the g subscript: V_{klnt} and

$V_{slot_{kilnt}}$ denote full-RTI loads on plant-bound trucks, and V_{klrt}^e denotes folded-RTI loads on supplier-bound trucks. Table 1 collects the complete nomenclature.

Table 1: Nomenclature of 3D-SCPTOP (stack-based reformulation)

Indices and sets	
G	Supplier and plant nodes, $G = \{s, p\}$.
N	Products, $n = 1, \dots, N$.
R	RTI types, $r = 1, \dots, R$.
H	Stack types, $h = 1, \dots, H$.
I	Stack-slot indices within a loading configuration, $i = 1, \dots, I $ (see eq. 1.1).
L	Loading configurations, $l = 1, \dots, L$.
K	Trucks, $k = 1, \dots, K$.
T	Planning periods, $t = 1, \dots, T$.
$M(r)$	Products carried by RTI type r , $M(r) \subseteq N$.
$R(h)$	RTI types compatible with stack type h , $R(h) \subseteq R$.
Parameters	
cK	Cost of using a truck.
lt	Lead transport time between plant and supplier (and vice versa).
i_{gn}	Initial inventory of $n \in N$ at $g \in G$ (full RTIs).
ie_{gr}	Initial inventory of folded RTIs $r \in R$ at $g \in G$.
ic_{gn}	Inventory cost of $n \in N$ at $g \in G$.
d_{gnt}	Production plan for $n \in N$ at $g \in G$ in $t \in T$.
q_{nt}	Initial arrivals of full RTIs of product n at p in $t \in \{1, \dots, lt\}$.
q_{rt}	Initial arrivals of folded RTIs of type r at s in $t \in \{1, \dots, lt\}$.
ss_{gnt}	Security stock of n at g in t (full RTIs).
ss_{gtr}	Security stock of folded RTIs r at g in t .
w_n	Weight of a full RTI carrying product n .
w_r^f	Weight of a folded RTI of type r .
pK^w	Maximum truck weight capacity.
$W^{\max}$	Per-stack crush-limit weight cap.
pK, qK, rK	Truck length, width, and height.
Γ_K	Truck volume, $\Gamma_K = pK \cdot qK \cdot rK$.
α	Minimum truck volume utilisation ($0 < \alpha \leq 1$).
p_h, q_h	Footprint length and width of stack type h .
r_{gr}	Height of one RTI of type r at node g (full at p , folded at s).
γ_{gr}	Volume of one RTI of type r at node g .
n_{hr}	RTIs of type $r \in R(h)$ per layer of stack type h .
M	A sufficiently large constant.
Decision variables – LC level	
Z_{gl}	$\in \{0, 1\}$. Equals 1 iff LC l is active in direction g .
X_{gihl}	$\in \{0, 1\}$. Equals 1 iff slot i of LC l is of stack type h in direction g .
L_{gihlr}	$\in \mathbb{Z}_{\geq 0}$. Number of layers of RTI type $r \in R(h)$ in slot i of LC l in direction g .
LX_{gihl}	$\in \{0, 1\}$. Equals 1 iff stack h 's length is along the x -axis in slot i .
LY_{gihl}	$\in \{0, 1\}$. Equals 1 iff stack h 's length is along the y -axis in slot i .
PX_{gil}, PY_{gil}	≥ 0 . Bottom-left coordinates of slot i in LC l in direction g .
A_{gijl}	$\in \{0, 1\}$. Equals 1 iff slot i is to the left of slot j in LC l .
B_{gijl}	$\in \{0, 1\}$. Equals 1 iff slot i is to the right of slot j in LC l .
C_{gijl}	$\in \{0, 1\}$. Equals 1 iff slot i is behind slot j in LC l .

D_{gijl}	$\in \{0, 1\}$. Equals 1 iff slot i is in front of slot j in LC l .
Decision variables – runtime level	
Y_{gkt}	$\in \{0, 1\}$. Equals 1 iff truck k is used in direction g in period t .
Y_{gklt}	$\in \{0, 1\}$. Equals 1 iff truck k uses LC l in direction g in period t .
$V_{kln t}$	$\in \mathbb{Z}_{\geq 0}$. Full RTIs of product n on plant-bound truck k using LC l in period t .
V_{kilnt}^{slot}	$\in \mathbb{Z}_{\geq 0}$. Full RTIs of product n in slot i of plant-bound truck k using LC l in period t .
V_{klrt}^e	$\in \mathbb{Z}_{\geq 0}$. Folded RTIs of type r on supplier-bound truck k using LC l in period t .
I_{gnt}	$\in \mathbb{Z}_{\geq 0}$. End-of-period inventory of n at g in t (full RTIs).
Ie_{gtr}	$\in \mathbb{Z}_{\geq 0}$. End-of-period inventory of folded RTIs r at g in t .
Q_{nt}	$\in \mathbb{Z}_{\geq 0}$. Full RTIs of product n in transit to the plant in t .
Q_{rt}	$\in \mathbb{Z}_{\geq 0}$. Folded RTIs of type r in transit to the supplier in t .

Objective

$$\text{Min } z = \sum_{g \in G} \sum_{k \in K} \sum_{t \in T} cK \cdot Y_{gkt} + \sum_{g \in G} \sum_{n \in N} \sum_{t \in T} ic_{gn} \cdot I_{gnt} \quad (1)$$

Inventory and flow balance

The inventory balance block is structurally identical to the original formulation. For completeness it is reproduced below.

$$I_{pnt} = I_{p,n,t-lt} - d_{pnt} + Q_{n,t-lt} \quad \forall n \in N, t \geq lt + 1 \quad (2)$$

$$I_{pnt} = I_{p,n,t-1} - d_{pnt} + q_{nt} \quad \forall n \in N, t \in \{2, \dots, lt\} \quad (3)$$

$$I_{pnt} = i_{pn} - d_{pnt} + q_{nt} \quad \forall n \in N, t = 1 \quad (4)$$

$$Ie_{ptr} = Ie_{p,t-1,r} + \sum_{n \in M(r)} d_{pnt} - Q_{rt} \quad \forall r \in R, t \geq 2 \quad (5)$$

$$Ie_{ptr} = ie_{pr} + \sum_{n \in M(r)} d_{pnt} - Q_{rt} \quad \forall r \in R, t = 1 \quad (6)$$

$$I_{snt} = I_{s,n,t-1} + d_{snt} - Q_{nt} \quad \forall n \in N, t \geq 2 \quad (7)$$

$$I_{snt} = i_{sn} + d_{snt} - Q_{nt} \quad \forall n \in N, t = 1 \quad (8)$$

$$Ie_{str} = Ie_{s,t-lt,r} + Q_{r,t-lt} - \sum_{n \in M(r)} d_{snt} \quad \forall r \in R, t \geq lt + 1 \quad (9)$$

$$Ie_{str} = Ie_{s,t-1,r} + q_{rt} - \sum_{n \in M(r)} d_{snt} \quad \forall r \in R, t \in \{2, \dots, lt\} \quad (10)$$

$$Ie_{str} = ie_{sr} + q_{rt} - \sum_{n \in M(r)} d_{snt} \quad \forall r \in R, t = 1 \quad (11)$$

$$I_{gnt} \geq ss_{gnt}, \quad Ie_{gtr} \geq ss_{gtr} \quad \forall g \in G, n \in N, r \in R, t \in T \quad (12)$$

Stack packing constraints

Exactly one stack type can be assigned to a slot, and only if its LC is active:

$$\sum_{h \in H} X_{ghl} \leq Z_{gl} \quad \forall g \in G, i \in I, l \in L. \quad (13)$$

The stack occupying slot i is oriented either with its length along the x -axis or along the y -axis:

$$LX_{ghl} + LY_{ghl} = X_{ghl} \quad \forall g \in G, h \in H, i \in I, l \in L. \quad (14)$$

The total stack height cannot exceed the truck height. The following constraint also forces $L_{ghlr} = 0$ whenever $X_{ghl} = 0$, since $r_{gr} > 0$:

$$\sum_{r \in R(h)} r_{gr} L_{ghlr} \leq rK \cdot X_{ghl} \quad \forall g \in G, h \in H, i \in I, l \in L. \quad (15)$$

Slots must not overlap. For each pair $i \neq j$ in LC l :

$$PX_{gil} + \sum_{h \in H} (p_h LX_{ghl} + q_h LY_{ghl}) \leq PX_{gjl} + M(1 - A_{gijl}) \quad (16)$$

$$PX_{gjl} + \sum_{h \in H} (p_h LX_{ghl} + q_h LY_{ghl}) \leq PX_{gil} + M(1 - B_{gijl}) \quad (17)$$

$$PY_{gil} + \sum_{h \in H} (q_h LX_{ghl} + p_h LY_{ghl}) \leq PY_{gjl} + M(1 - C_{gijl}) \quad (18)$$

$$PY_{gjl} + \sum_{h \in H} (q_h LX_{ghl} + p_h LY_{ghl}) \leq PY_{gil} + M(1 - D_{gijl}) \quad (19)$$

with the cover constraint ensuring that whenever both slots are used at least one relative-position variable is active:

$$A_{gijl} + B_{gijl} + C_{gijl} + D_{gijl} \geq \sum_h X_{ghl} + \sum_h X_{ghl} - 1 \quad \forall g, l, i \neq j. \quad (20)$$

Containment in the truck floor:

$$PX_{gil} + \sum_{h \in H} (p_h LX_{ghl} + q_h LY_{ghl}) \leq pK + M(1 - \sum_h X_{ghl}) \quad \forall g, i, l, \quad (21)$$

$$PY_{gil} + \sum_{h \in H} (q_h LX_{ghl} + p_h LY_{ghl}) \leq qK + M(1 - \sum_h X_{ghl}) \quad \forall g, i, l. \quad (22)$$

Minimum truck volume utilisation, measured on the actual RTI volumes (kept in RTI-volume form so non-full layers or fillers can be introduced in later extensions):

$$\sum_{i \in I} \sum_{h \in H} \sum_{r \in R(h)} \gamma_{gr} n_{hr} L_{ghlr} \geq \alpha \Gamma_K Z_{gl} \quad \forall g \in G, l \in L. \quad (23)$$

Truck-LC linking and runtime loading

A truck uses at most one LC in a period, and can only use an active LC:

$$\sum_{l \in L} Y_{gklt} = Y_{gkt} \quad \forall g, k, t \quad (24)$$

$$Y_{gklt} \leq Z_{gl} \quad \forall g, k, l, t \quad (25)$$

For plant-bound trucks, full RTIs of type r loaded in slot i cannot exceed the layer capacity of the stack placed in that slot; products sharing the same RTI type partition that slot-capacity:

$$\sum_{n \in M(r)} V_{kilnt}^{\text{slot}} \leq \sum_{h: r \in R(h)} n_{hr} L_{p,i,h,l,r} + M(1 - Y_{pkt}) \quad \forall k, i, l, r, t. \quad (26)$$

Truck-level aggregation of per-slot loads:

$$V_{klnt} = \sum_{i \in I} V_{kilnt}^{\text{slot}} \quad \forall k, l, n, t. \quad (27)$$

For supplier-bound trucks, folded RTIs of type r are capped by the aggregate layer capacity of the LC in use (no slot-level tracking is needed because folded-RTI weight is deterministic in r):

$$V_{klrt}^e \leq \sum_{i \in I} \sum_{h: r \in R(h)} n_{hr} L_{s,i,h,l,r} + M(1 - Y_{skt}) \quad \forall k, l, r, t. \quad (28)$$

Weight constraints

Per-truck weight, plant-bound:

$$\sum_{l \in L} \sum_{n \in N} w_n V_{klnt} \leq pK^w Y_{pkt} \quad \forall k, t. \quad (29)$$

Per-truck weight, supplier-bound:

$$\sum_{l \in L} \sum_{r \in R} w_r^f V_{klrt}^e \leq pK^w Y_{skt} \quad \forall k, t. \quad (30)$$

Per-stack crush-limit, period-exact on plant-bound trucks (the cap reflects the actual product mix loaded in each slot):

$$\sum_{n \in N} w_n V_{kilnt}^{\text{slot}} \leq W^{\max} \quad \forall k, i, l, t. \quad (31)$$

Per-stack crush-limit on supplier-bound trucks (deterministic in the LC since folded-RTI weight depends only on r):

$$\sum_{h \in H} \sum_{r \in R(h)} w_r^f n_{hr} L_{s,i,h,l,r} \leq W^{\max} \quad \forall i, l. \quad (32)$$

Flow aggregation

$$Q_{nt} = \sum_{k \in K} \sum_{l \in L} V_{klnt} \quad \forall n, t \quad (33)$$

$$Q_{rt} = \sum_{k \in K} \sum_{l \in L} V_{klrt}^e \quad \forall r, t \quad (34)$$

1.1 Symmetry breaking and slot-set sizing

The slots $i \in I$ of a given LC are interchangeable, so any solution admits $|I|!$ permutations of the slot index. We break this symmetry by enforcing that slot $i + 1$ is used only if slot i is used:

$$\sum_{h \in H} X_{g,i+1,h,l} \leq \sum_{h \in H} X_{g,i,h,l} \quad \forall g \in G, l \in L, i \in \{1, \dots, |I| - 1\}. \quad (35)$$

The cardinality of I is a modelling parameter. An upper bound that eliminates provably unused slot variables without cutting any feasible layout is

$$|I| \leq \left\lfloor \frac{pK \cdot qK}{\min_{h \in H} p_h \cdot q_h} \right\rfloor, \quad (36)$$

corresponding to tiling the truck floor with the smallest-footprint stack type; it serves as a tight preprocessing limit on $|I|$.

Domains

$$\begin{aligned} Z_{gl}, X_{gihl}, LX_{gihl}, LY_{gihl}, A_{gijl}, B_{gijl}, C_{gijl}, D_{gijl}, Y_{gkt}, Y_{gklt} &\in \{0, 1\}, \\ L_{gihlr}, V_{klnt}, V_{kilnt}^{\text{slot}}, V_{klrt}^e, I_{gnt}, I_{egtr}, Q_{nt}, Q_{rt} &\in \mathbb{Z}_{\geq 0}, \\ PX_{gil}, PY_{gil} &\geq 0. \end{aligned}$$

2 Branch and Price Solution Approach

There are three main resolution issues with the 3D-SCPTOP model of Section 1. The first one is that a priori we do not know how many loading configurations are needed, so $|L|$ must be overestimated; every configuration that turns out to be unnecessary still carries its full block of placement variables. The second is that the model replicates a complete two-dimensional packing problem $|G| \cdot |L|$ times, and each replica contributes $4 \binom{|I|}{2}$ big- M disjunctions (16)–(19) whose linear relaxation is very weak. The third is that the configurations are mutually interchangeable, so the search tree is massively symmetric. Together these make a direct MILP solution impractical beyond toy sizes. Therefore, a column generation scheme based on branch and price is developed.

The key components of the branch and price algorithm are the master problem (MP), the restricted master problem (RMP), the pricing problem (PP) and the branching policy. The MP is a reformulation of the model that exploits its decomposable structure. Inspecting Section 1, every geometric constraint (13)–(23) carries a single configuration index l , and every loading constraint (26)–(32) carries a single truck index k and a single period t ; the transport and inventory block sees them *only* through the aggregate flows Q_{nt} and Q_{rt} of (33)–(34). The model is therefore block-angular, with one independent block per dispatched truck. This suggests taking as the unit of decision neither the individual RTI nor the loading configuration, but the *complete loaded truck*.

The MP is typically too large to be solved directly because it involves a huge number of columns representing all possible truck loads. To manage this complexity we consider only a smaller subset of columns, which leads to the RMP. Solving the RMP provides an optimum for the included subset, but not necessarily for the entire column set. RMP integrality constraints are therefore relaxed to obtain dual variables, and the PP uses those duals to identify columns of negative reduced cost that should be added to the RMP. If no such column exists, the

integrality-relaxed MP has been solved to optimality. If all the variables associated with the solution are integers, an integer solution to the current branch has been found. If not, the branch must be divided by adding further constraints. Figure 1 exemplifies the algorithm.

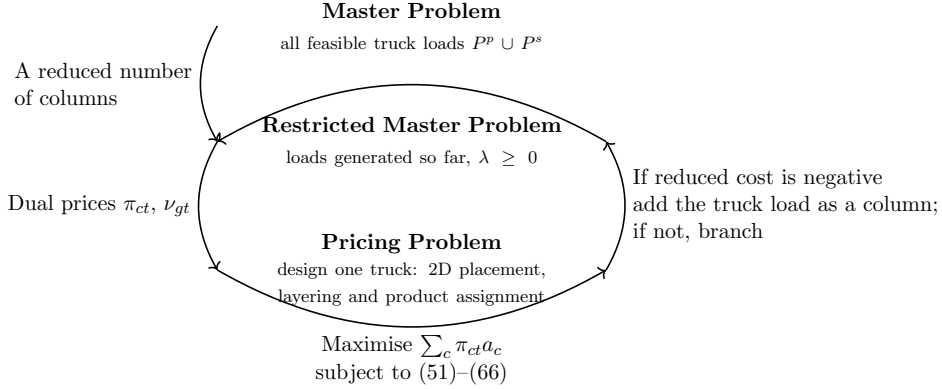


Figure 1: Branch and price scheme for 3D-SCPTOP. Every geometric, height, crush-limit, payload and full-truckload constraint is enforced inside the pricing problem, so each column is a physically loadable truck.

2.1 Formulation of the Master Problem and the Restricted Master Problem

The key idea of the set-partitioning formulation for 3D-SCPTOP is to enumerate all the feasible *truck loads* with their associated costs. A plant-bound truck load is a vector $\mathbf{a} = (a_n)_{n \in N}$ of full RTIs for which there exist a slot count $m \leq |I|$, stack types $h_1, \dots, h_m \in H$, orientations, bottom-left coordinates (x_i, y_i) , layer counts λ_{ir} and per-slot product loads v_{in} such that

$$\text{the } m \text{ footprints, rotated as chosen, do not overlap and lie inside } pK \times qK, \quad (37)$$

$$\sum_{r \in R(h_i)} r_{pr} \lambda_{ir} \leq rK \quad \forall i \leq m, \quad (38)$$

$$\sum_{n \in M(r)} v_{in} = n_{h_i r} \lambda_{ir} \quad \forall i \leq m, r \in R(h_i), \quad (39)$$

$$\sum_{n \in N} w_n v_{in} \leq W^{\max} \quad \forall i \leq m, \quad (40)$$

$$\sum_{n \in N} w_n a_n \leq pK^w, \quad \frac{\sum_n \gamma_{pp(n)} a_n}{\Gamma_K} + \frac{\sum_n w_n a_n}{pK^w} \geq \alpha, \quad (41)$$

$$a_n = \sum_{i \leq m} v_{in} \quad \forall n \in N. \quad (42)$$

Let P^p be the set of all such loads, and P^s the analogous set of supplier-bound loads $\mathbf{b} = (b_r)_{r \in R}$, built with folded heights r_{sr} , folded weights w_r^f and folded volumes γ_{sr} ; there no product assignment is needed, because the weight of a folded RTI depends only on its type.

Two features of this definition matter. First, a column carries the *per-slot* product assignment v_{in} , so the crush limit (40) is evaluated on the product mix actually riding in that slot. The column therefore reproduces the period-exact cap (31) of the compact model, and not the conservative surrogate $\bar{w}_r = \max_{n \in M(r)} w_n$ that a pool of capacity-only configurations is forced to use. Second, a column is a feasibility certificate: the placement $(h_i, x_i, y_i, \text{rotation})$ witnessing $\mathbf{a} \in P^p$ is retained, so any solution can be handed to a loading crew as a drawing and re-verified independently of the solver that produced it.

Constraint (41) deserves a comment. The full-truckload requirement is imposed on the *binding* resource: a truck counts as full once it reaches a fraction α of either the trailer volume

or the payload. Imposing it on volume alone, as (23) does, makes dense freight structurally infeasible — filling α of the volume with heavy RTIs can exceed pK^w , so no truck is dispatchable at all and the instance is empty. Section 3 shows that both regimes occur across the benchmark.

Let $\lambda_{pt}^g \in \mathbb{Z}_{\geq 0}$ count the trucks dispatched in direction g and period t carrying load p , and let a_n^p and a_r^p be the RTIs of product n , respectively of type r , that load p contains. Every truck costs cK . The MP of our problem is

$$\min \quad cK \sum_{g \in G} \sum_{t \in T} \sum_{p \in P^g} \lambda_{pt}^g + \sum_{g \in G} \sum_{n \in N} \sum_{t \in T} ic_{gn} I_{gnt} \quad (43)$$

subject to

$$\sum_{p \in P^p} a_n^p \lambda_{pt}^p = Q_{nt} \quad \forall n \in N, t \in T \quad (44)$$

$$\sum_{p \in P^s} a_r^p \lambda_{pt}^s = Q_{rt} \quad \forall r \in R, t \in T \quad (45)$$

$$\sum_{p \in P^g} \lambda_{pt}^g \leq |K| \quad \forall g \in G, t \in T \quad (46)$$

(2)–(12) (inventory balances and coverage stock)

$$\lambda_{pt}^g \in \mathbb{Z}_{\geq 0} \quad \forall g \in G, p \in P^g, t \in T.$$

Objective (43) minimises transport plus inventory holding cost. Constraints (44)–(45) state that the flow of full and folded RTIs is exactly what the dispatched trucks carry, and (46) caps the fleet available per direction and period.

MP and 3D-SCPTOP have the same optimal value whenever $|L|$ is not restricted a priori. Given a 3D-SCPTOP solution, each used truck (k, t) induces a load vector satisfying (37)–(42), obtained by reading the slot data off its configuration and setting $v_{in} = V_{kilnt}^{\text{slot}}$; counting trucks by load vector gives a MP solution of equal cost. Conversely, introducing one configuration per distinct load used by a MP solution and assigning trucks accordingly reconstructs a 3D-SCPTOP solution of the same cost. This equivalence is what distinguishes the present approach from a pool-based method such as PLC-SCPTOP: a predefined pool is a *restriction* and can only yield upper bounds of unknown quality, whereas MP is the same problem written differently, so any bound derived from it is a bound on the original.

The RMP replaces P^g by a subset $\bar{P}^g$ and relaxes integrality,

$$\min \quad cK \sum_{g \in G} \sum_{t \in T} \sum_{p \in \bar{P}^g} \lambda_{pt}^g + \sum_{g \in G} \sum_{n \in N} \sum_{t \in T} ic_{gn} I_{gnt} \quad (47)$$

subject to (44)–(46) restricted to $\bar{P}^g$, with $\lambda_{pt}^g \geq 0$. Dual values are obtained by solving the RMP. Let π_{nt} and π_{rt} be the duals of (44)–(45) and $\nu_{gt} \leq 0$ those of (46). The reduced cost associated with a plant-bound load $p \in P^p$ dispatched in period t is

$$\bar{c}(p, t) = cK - \sum_{n \in N} \pi_{nt} a_n^p - \nu_{pt}, \quad (48)$$

and analogously in the supplier direction. A load with negative reduced cost can improve the current RMP solution. For each iteration we solve the subproblem of finding the load of minimum reduced cost; loads with negative reduced cost are added as new columns, the RMP is re-solved to generate new duals, and the process repeats until no further improving column is discovered.

2.2 Lower bounds

Care is needed here, because the natural candidate is not a bound at all. The RMP is a *restriction* of the MP: it optimises over $\bar{P}^g \subseteq P^g$, so $z_{\text{RMP}} \geq z_{\text{MP}}$. Reporting the RMP value as a lower bound is therefore valid only once column generation has converged, and wrong before then. We use two bounds accordingly.

Certified bound. When the PP has *proven* that $\zeta_{gt} := \max_{p \in P^g} \sum_c \pi_{ct} a_c^p \leq cK - \nu_{gt}$ for every g and t , no column outside $\bar{P}^g$ has negative reduced cost, so the current duals are feasible for the dual of the full MP. Weak duality then gives $z_{\text{RMP}} = z_{\text{MP}} \leq z_{\text{3D-SCPTOP}}^*$. We report such bounds as *certified*; the gap against the final integer solution is then a genuine optimality gap, and not a gap with respect to a pool.

Lagrangian bound. Certification requires the two-dimensional packing subproblem to be closed to proven optimality, which is not always possible inside a sensible time limit. Dualising (44)–(45) with the current π leaves a subproblem in which each column enters with reduced cost (48); since the number of trucks an optimal solution can dispatch in total is bounded by $\bar{X}$, the correction

$$z_{\text{MP}} \geq z_{\text{RMP}} + \bar{X} \cdot \min_{g \in G, t \in T} \min\{0, cK - \zeta_{gt} - \nu_{gt}\} \quad (49)$$

holds at every iteration, converged or not. Two choices make (49) usable rather than vacuous. The value substituted for ζ_{gt} is the solver’s own dual bound on the PP, never its incumbent, so a subproblem stopped early still yields a valid verdict. And $\bar{X}$ must be a genuine bound on the fleet actually used: the textbook form charges $|K|$ separately for every direction and period, which here is roughly an order of magnitude too pessimistic and drove the reported gaps above 80%. Because transport cost alone cannot exceed the objective, any incumbent $\bar{z}$ gives $\bar{X} = \bar{z}/cK$, which is both valid and tight enough to be informative.

Rounding the duals. The prices π are irrational in general and must be rounded before entering an integer-coefficient subproblem. We round them *upwards*, so the rounded objective dominates the true one and the returned bound remains an over-estimate of ζ_{gt} , as both bounds above require. Rounding to nearest — the obvious choice — makes the bound a slight under-estimate and silently invalidates the certificate; in our first implementation it returned 1000.1 against a threshold of 1000.0 on instances where the reduced cost was exactly zero, and convergence was never declared.

2.3 Pricing Subproblem

We solve the pricing problem with a constraint programming approach. The indices and parameters are the same as in the model proposed before, but the decision variables do not depend on the configuration index. Let c range over products when $g = p$ and over RTI types when $g = s$, let $\rho(c)$ be the RTI type carrying c , and let S be the slot bound (36). Table 2 describes the new decision variables.

Table 2: Nomenclature of the pricing subproblem

Decision variables

u_i	Binary variable indicating whether slot i is used in the new load
χ_{ih}	Binary variable indicating whether slot i holds a stack of type h
ϱ_{ih}	Binary variable indicating whether that stack is rotated by 90°
δ_i^x, δ_i^y	Footprint of slot i along the x and y axes
x_i, y_i	Bottom-left coordinates of slot i on the truck floor
ℓ_{ir}	Number of layers of RTI type r in slot i
v_{ic}	Number of RTIs of commodity c loaded in slot i
a_c	Number of RTIs of commodity c in the new load
θ	Binary variable indicating whether the full-truckload requirement is met on volume rather than on weight

The objective is to find new truck loads with negative reduced cost. Dropping the constant $cK - \nu_{gt}$ of (48), that is

$$\max z = \sum_c \pi_{ct} a_c \quad (50)$$

subject to

$$\sum_{h \in H} \chi_{ih} = u_i, \quad \varrho_{ih} \leq \chi_{ih} \quad \forall i \leq S, h \in H \quad (51)$$

$$\delta_i^x = \sum_h (p_h \chi_{ih} + (q_h - p_h) \varrho_{ih}) \quad \forall i \leq S \quad (52)$$

$$\delta_i^y = \sum_h (q_h \chi_{ih} + (p_h - q_h) \varrho_{ih}) \quad \forall i \leq S \quad (53)$$

$$\text{NoOVERLAP2D} \left(\{ [x_i, x_i + \delta_i^x] \times [y_i, y_i + \delta_i^y] \}_{i: u_i=1} \right) \quad (54)$$

$$x_i + \delta_i^x \leq pK, \quad y_i + \delta_i^y \leq qK \quad \forall i \leq S \quad (55)$$

$$\sum_{i \leq S} \sum_h p_h q_h \chi_{ih} \leq pK \cdot qK \quad (56)$$

$$\ell_{ir} \leq \lfloor rK/r_{gr} \rfloor \sum_{h: r \in R(h)} \chi_{ih} \quad \forall i \leq S, r \in R \quad (57)$$

$$\sum_{r \in R} r_{gr} \ell_{ir} \leq rK \quad \forall i \leq S \quad (58)$$

$$\sum_{c: \rho(c)=r} v_{ic} = n_{hr} \ell_{ir} \quad \forall i \leq S, r \in R \quad (59)$$

$$\sum_c w_c v_{ic} \leq W^{\max} \quad \forall i \leq S \quad (60)$$

$$a_c = \sum_{i \leq S} v_{ic} \quad \forall c \quad (61)$$

$$\sum_c w_c a_c \leq pK^w \quad (62)$$

$$\sum_c \gamma_{g\rho(c)} a_c \geq \alpha \Gamma_K \quad \text{if } \theta = 1 \quad (63)$$

$$\sum_c w_c a_c \geq \alpha pK^w \quad \text{if } \theta = 0 \quad (64)$$

$$\sum_{i \leq S} \sum_h p_h q_h \chi_{ih} \geq \alpha pK \cdot qK \quad \text{if } \theta = 1 \quad (65)$$

$$\sum_h h \chi_{ih} \geq \sum_h h \chi_{i+1,h} \quad \forall i < S. \quad (66)$$

Constraints (51)–(61) restrict the loads used in the pricing problem to feasible ones, reproducing (37)–(42). Equation (56) is a redundant area cut, and (65) its counterpart under the volume regime: a slot contributes at most (footprint area) $\times rK$ of RTI volume, so the utilisation floor implies a floor on the occupied floor area. The latter is what allows the solver to find a *first* feasible layout quickly, which for tight instances is harder than optimising once one is known. Constraint (66) breaks the symmetry among interchangeable slots by ordering them on the stack-type index.

The subproblem is stated as a constraint program because its hard core is the disjunctive non-overlap relation (54), which a CP solver propagates natively through a global constraint, whereas a MILP must encode it with the four big- M disjunctions (16)–(19) and accept the weak relaxation they entail. Section 3 reports the resulting difference.

2.4 Initial Columns

Column generation converges from any starting set, but a good one shortens it and leaves the integer master more freedom to combine loads. We initialise $\bar{P}^g$ by solving (50)–(66) under a family of demand-derived price vectors rather than duals: the unit vector, which maximises the RTI count; every single-commodity vector $\mathbf{e}_c$, which yields pure loads and keeps the master flexible; the aggregate demand mix and each per-period demand mix; and a set of random draws from the simplex for diversification.

Two devices make this robust. A constructive *grid layout* — tile the floor with a single stack type in its better orientation and fill each slot to its height, weight and crush limit — is supplied to the CP solver as a hint and retained as a fallback column. For the near-tiling footprints of a real RTI catalogue it is already close to the best attainable utilisation, and it guarantees that no direction is ever left without columns. And any pricing call that returns nothing is retried with a longer limit before being believed, since an empty answer usually means the solver was starved of time rather than that no load exists.

Before the integer master is solved, the pool is enriched with *reduced* loads obtained by unloading RTIs from existing columns. Removing RTIs from a valid layout cannot break the geometry, the height budget, a crush limit or the payload, so every reduced load that still clears the full-truckload floor is itself feasible. This matters because columns priced at the LP optimum are close to full trucks, and integer combinations of full trucks are too coarse to land inside the narrow delivery band left open by the supplier’s coverage stock. The LP hides this, because it can interpolate between columns; the integer master cannot.

2.5 Branching

Here we use the depth-first branching rule: once no new promising column can be found through the pricing problem, we take the λ_{pt}^g with the largest fractional part and fix it to its ceiling, that is, we fix a truck load that must be dispatched. If rounding up renders the node infeasible, we round the same variable down instead. After fixing the variable, more columns are generated by the pricing algorithm, so new truck loads keep being created *after* the branching decisions — which a price-and-branch scheme over a frozen pool cannot do. Repeating this strategy leads to an integer solution of the RMP. As in Moreno, Mula, and Poler 2024, this method of branching does not guarantee an optimal solution, but it does guarantee a feasible one; the certified bound of Section 2.1 is what quantifies the remaining loss.

2.6 Two corrections to the compact model

Implementing 3D-SCPTOP surfaced two defects worth recording, both invisible at $lt = 1$.

Inventory recursion under a lead time. Constraints (2) and (9) carry inventory forward from period $t - lt$, whereas inventory accumulates one period at a time and only the *arrival* is delayed. They must read

$$I_{pnt} = I_{p,n,t-1} - d_{pnt} + Q_{n,t-lt} \quad \forall n \in N, t \geq lt + 1, \quad (67)$$

$$I_{estr} = I_{e,s,t-1,r} + Q_{r,t-lt} - \sum_{n \in M(r)} d_{snt} \quad \forall r \in R, t \geq lt + 1. \quad (68)$$

With $lt = 1$ the two versions coincide, which is why the defect survived; with $lt \geq 2$ the original silently discards $lt - 1$ periods of stock.

Algorithm 1 Branch and price for 3D-SCPTOP

```
1:  $\bar{P} \leftarrow \text{INITIALCOLUMNS}()$  ▷ Section 2.4
2:  $sol_{int} \leftarrow \text{False}$ 
3: while  $sol_{int} == \text{False}$  do
4:    $rc \leftarrow -1$ 
5:   while  $rc < 0$  do
6:      $\pi, \nu, \lambda \leftarrow \text{SOLVERMP}(\bar{P})$ 
7:      $rc, p \leftarrow \text{PRICE}(\pi, \nu, \tau_{\text{fast}})$ 
8:     if  $rc \geq 0$  then
9:        $rc, p \leftarrow \text{PRICE}(\pi, \nu, \tau_{\text{exact}})$  ▷ a fast pass may simply have missed a load
10:    end if
11:    if  $rc < 0$  then
12:      Append  $p$  to  $\bar{P}$ 
13:    else if the pricing bound proves  $rc \geq 0$  then
14:       $\text{LB} \leftarrow \text{RMP value}$  ▷ certified, Section 2.1
15:    end if
16:  end while
17:  if  $\lambda$  is integral then
18:     $sol_{int} \leftarrow \text{True}$ ; break
19:  end if
20:   $(p, t) \leftarrow \arg \max \text{ fractional part of } \lambda_{pt}^g$ 
21:  fix  $\lambda_{pt}^g \geq \lceil \lambda_{pt}^g \rceil$ , or  $\leq \lfloor \lambda_{pt}^g \rfloor$  if that node is infeasible
22: end while
23:  $\bar{P} \leftarrow \bar{P} \cup \text{REDUCEDLOADS}(\bar{P})$ 
24:  $\text{UB} \leftarrow \text{SOLVERMP}(\bar{P})$  as an integer program
25: verify every dispatched column against (37)–(42)
26: return (UB, LB)
```

Coverage stock on folded RTIs. Requiring $Ie_{gtr} \geq ss_{gtr}$ at *both* nodes, as (12) does, forces the plant to hoard empties it has no use for and can render an otherwise feasible instance infeasible when the RTI fleet is tight. The operationally meaningful requirement is that the *supplier* hold enough folded RTIs to cover its production, so we impose the bound at $g = s$ only.

3 Computational Experiments

The purpose of the experiments is not to establish optimality. For procurement transport planning at an operational level, what matters is whether a procedure returns, in a usable amount of time, a plan that a loading crew can actually execute and that improves on what the company does today. The experiments are organised around those two questions, and a third that turns out to dominate both: whether a model that abstracts the loading geometry away produces plans that can be executed at all.

All calculations use Python. The master and restricted master problems, the volumetric benchmark and the compact model are solved with Gurobi 12; the pricing subproblem (50)–(66) with CP-SAT from OR-Tools 9.12. Tests run on a 16-core machine with 30 GB of RAM. A relative gap of 2% is set on every integer program.

3.1 Instance generation

Thirty-six instances are artificially generated as a full factorial over the four explanatory variables of the study: the number of products $|N| \in \{4, 8\}$, the horizon $|T| \in \{5, 10\}$, the number of RTI types $|R| \in \{2, 3, 4\}$ and the volumetric granularity $v \in \{60, 90, 120\}$. Instances are named $I_{-}|N|_{-}|T|_{-}|R|_{-}v$.

- **Container dimensions:** footprints are drawn from the industrial catalogue $\{1200 \times 800, 1200 \times 1000, 1200 \times 1600, 2400 \times 1200\}$ mm. Heights are quantised to 50 mm and confined to $[400, 1500]$ mm so that at least two layers fit under the trailer roof. Whenever $|R| \geq 3$ several RTI types share a footprint, reproducing a real catalogue in which container families sit on a common pallet base; this is what allows a stack to mix types across layers.
- **Container volume:** v is the number of erected RTIs that fill one truck by volume, and every RTI type is generated with volume at least Γ_K/v . Larger v means smaller RTIs and a harder placement subproblem.
- **Demand:** generated per product and period by deviating $\pm 10\%$ from a mean calibrated so that roughly 2.5 truckloads are consumed per period. Calibrating the mean keeps difficulty comparable across the factorial instead of letting it drift with the RTI size.
- **Weights:** the content weight of a full RTI scales with its volume and varies by $\pm 25\%$ between products sharing an RTI type, which is what makes the per-stack crush limit depend on the period's product mix rather than on the RTI type alone.

The truck is a standard semi-trailer, $pK \times qK \times rK = 13600 \times 2400 \times 3000$ mm, payload $pK^w = 24$ t, per-stack crush limit $W^{\max} = 1200$ kg, minimum utilisation $\alpha = 0.55$ and cost $cK = 1000$ per dispatch. Lead time is $lt = 1$ and coverage stock one period of demand.

Sizing the RTI fleet. One generation parameter deserves comment because it is easy to get wrong and the symptom is misleading. Empties return by the truckload, and a return truck leaves only once the plant has accumulated enough folded RTIs to clear the full-truckload floor. Since folded RTIs pack very densely — two to three hundred to a trailer against an accumulation rate of a few dozen per period — that takes several periods, during which the supplier must live off its own folded stock. If the fleet is sized only for the lead time, the linear relaxation remains feasible by dispatching fractional trucks while the integer problem has no solution at all. We therefore size the supplier's folded stock to cover $lt + SC + \lceil \alpha \Gamma_K / (\gamma_{sr} \cdot \text{empties per period}) \rceil$ periods, which is the standard closed-loop argument that an RTI fleet must cover the cycle time.

3.2 Methods compared

- **MATH-3D-SCPTOP** — Algorithm 1.
- **Company procedure (PLC-SCPTOP)** — the master problem over a fixed pool of manually derived loading configurations, one regular grid per container type, as produced today by a spreadsheet.
- **SCPTOP** — the volumetric benchmark of Appendix ??, identical to the master in every respect *except* that truck capacity is volumetric. The comparison therefore isolates the effect of the loading representation.

- **Compact 3D-SCPTOP** — solved directly, on the instances where that is still possible, to measure the matheuristic’s true optimality gap without a bounding argument.

Every truck that MATH-3D-SCPTOP dispatches is re-verified from scratch against (37)–(42) by a routine that shares no code with the solver that produced it. Every truck that SCPTOP dispatches is submitted to an independent constraint programme asking whether that exact set of RTIs can be stacked and placed in a trailer at all.

3.3 Results

Table 3: MATH-3D-SCPTOP against the volumetric benchmark on the 36 generated instances. *unloadable* counts the truck loads SCPTOP dispatches that a constraint programme proves cannot be placed in a trailer, out of all loads it dispatches. Δz is the cost of MATH-3D-SCPTOP relative to SCPTOP.

Instance	MATH-3D-SCPTOP				SCPTOP (volumetric)					Δz
	z	trucks	loadable	time (s)	z	trucks	unloadable	loadable	time (s)	
I.4.10.2.120	62 545	45	all	926	53 441	36	31 / 36	0	0	17.0%
I.4.10.2.60	44 033	34	all	1 034	31 910	22	20 / 22	0	0	38.0%
I.4.10.2.90	45 331	29	all	910	42 315	26	19 / 26	0	0	7.1%
I.4.10.3.120	58 104	42	all	1 196	47 916	32	24 / 32	0	0	21.3%
I.4.10.3.60	35 401	24	all	970	30 909	20	18 / 20	0	1	14.5%
I.4.10.3.90	45 443	30	all	962	44 564	30	6 / 30	0	4	2.0%
I.4.10.4.120	51 983	36	all	1 076	50 693	35	15 / 35	0	0	2.5%
I.4.10.4.60	48 190	36	all	1 056	36 093	24	23 / 24	0	0	33.5%
I.4.10.4.90	43 703	29	all	1 087	40 776	26	15 / 26	0	0	7.2%
I.4.5.2.120	28 220	19	all	1 200	25 176	16	16 / 16	0	0	12.1%
I.4.5.2.60	14 728	9	all	1 204	13 802	8	7 / 8	0	0	6.7%
I.4.5.2.90	19 747	13	all	927	17 788	11	8 / 11	0	0	11.0%
I.4.5.3.120	26 118	16	all	912	25 880	16	8 / 16	0	0	0.9%
I.4.5.3.60	19 657	15	all	1 203	13 627	9	8 / 9	0	0	44.3%
I.4.5.3.90	21 715	14	all	901	21 502	14	0 / 14	0	0	1.0%
I.4.5.4.120	28 278	17	all	1 202	26 293	15	11 / 15	0	1	7.6%
I.4.5.4.60	16 204	12	all	906	12 985	9	8 / 9	0	2	24.8%
I.4.5.4.90	21 633	14	all	912	20 568	13	9 / 13	0	1	5.2%
I.8.10.2.120	63 715	44	all	1 141	55 521	37	29 / 37	0	1	14.8%
I.8.10.2.60	44 579	35	all	1 082	31 457	22	20 / 22	0	600	41.7%
I.8.10.2.90	49 436	35	all	1 084	43 359	30	27 / 30	0	0	14.0%
I.8.10.3.120	58 384	40	all	1 200	52 682	35	24 / 35	0	0	10.8%
I.8.10.3.60	32 948	24	all	1 210	28 932	20	18 / 20	0	1	13.9%
I.8.10.3.90	41 541	27	all	1 047	37 722	24	20 / 24	0	0	10.1%
I.8.10.4.120	69 149	46	all	1 201	58 336	37	30 / 37	0	0	18.5%
I.8.10.4.60	44 847	35	all	1 200	30 123	21	21 / 21	0	0	48.9%
I.8.10.4.90	48 150	31	all	1 200	43 311	27	15 / 27	0	1	11.2%
I.8.5.2.120	27 196	17	all	1 200	26 165	16	5 / 16	0	0	3.9%
I.8.5.2.60	21 378	16	all	1 200	15 165	10	9 / 10	0	0	41.0%
I.8.5.2.90	20 200	13	all	1 039	18 814	12	0 / 12	0	0	7.4%
I.8.5.3.120	23 547	15	all	938	21 053	13	8 / 13	0	0	11.8%
I.8.5.3.60	21 342	16	all	924	14 153	9	9 / 9	0	0	50.8%
I.8.5.3.90	20 940	14	all	1 200	18 650	12	0 / 12	0	0	12.3%
I.8.5.4.120	25 805	16	all	917	24 363	15	5 / 15	0	0	5.9%
I.8.5.4.60	18 772	13	all	938	13 585	8	8 / 8	0	0	38.2%
I.8.5.4.90	22 101	14	all	1 017	20 996	13	9 / 13	0	0	5.3%

Table 4: Results by instance factor. *unloadable* is the share of SCPTOP truck loads proven impossible to place.

Group	trucks (MATH)	trucks (SCPTOP)	unloadable	Δz	columns	time (s)
$ N = 4$	24.1	20.1	68%	14.3%	76	1 033
$ N = 8$	25.1	20.1	71%	20.0%	124	1 097
$ T = 5$	14.6	12.2	58%	16.1%	96	1 041
$ T = 10$	34.6	28.0	74%	18.2%	104	1 088
$ R = 2$	25.8	20.5	78%	17.9%	87	1 079
$ R = 3$	23.1	19.5	61%	16.1%	110	1 055
$ R = 4$	24.9	20.2	70%	17.4%	104	1 059
$v = 60$	22.4	15.2	93%	33.0%	71	1 077
$v = 90$	21.9	19.8	54%	7.8%	115	1 024
$v = 120$	29.4	25.2	68%	10.6%	114	1 093
all	24.6	20.1	70%	17.1%	100	1 065

Table 5: Case study on the industrial RTI catalogue. *company* is PLC-SCPTOP over the configurations the planner derives from average demand mixes; *benchmark* is PLC-SCPTOP over a richer hand-drawn set (regular grids and two-zone layouts). A dash means the procedure could not serve the demand programme with its configurations.

Scenario	Instance	company		benchmark		MATH-3D-SCPTOP		saving	
		z	trucks	z	trucks	z	trucks	cost	trucks
low	CASE_low_1	89 790	64	88 852	63	79 517	53	11.44%	17.19%
low	CASE_low_2	—	—	112 801	88	83 325	58	—	—
low	CASE_low_3	—	—	—	—	40 976	28	—	—
	mean low							11.4%	17.2%
normal	CASE_normal_1	50 461	40	49 549	39	43 979	33	12.84%	17.5%
normal	CASE_normal_2	124 549	87	125 430	88	109 786	72	11.85%	17.24%
normal	CASE_normal_3	—	—	—	—	51 510	41	—	—
	mean normal							12.3%	17.4%
high	CASE_high_1	52 166	40	49 201	37	42 405	30	18.71%	25.0%
high	CASE_high_2	78 290	58	79 195	59	72 090	51	7.92%	12.07%
high	CASE_high_3	115 988	89	115 974	89	102 311	74	11.79%	16.85%
	mean high							12.8%	18.0%

Table 6: MATH-3D-SCPTOP against the compact 3D-SCPTOP model, which is warm-started with the matheuristic’s plan. The compact model is given 30 minutes and $|L| = 3$ configurations per direction; *true gap* is the matheuristic’s distance from the compact model’s incumbent.

Instance	compact 3D-SCPTOP					MATH-3D-SCPTOP		
	z	gap	vars	cons	time (s)	z	time (s)	true gap
S_2.2.2	3 219	0.0%	11 908	14 910	3	4 248	225	31.97%
S_2.2.3	3 013	0.0%	12 262	15 914	1	3 103	215	3.00%
S_2.3.2	6 947	0.0%	12 672	16 042	2	7 968	234	14.69%
S_2.3.3	6 982	0.1%	19 173	25 324	14	7 004	144	0.32%
S_3.2.2	3 165	0.0%	12 586	14 944	3	3 171	226	0.21%
S_3.2.3	3 504	0.0%	19 072	23 550	2	3 529	225	0.73%
S_3.3.2	7 230	0.6%	13 689	16 093	3	8 173	267	13.05%
S_3.3.3	6 137	0.2%	20 442	25 375	28	6 202	170	1.06%

Table 7: Pricing technology. The same matheuristic is run with the subproblem solved as a constraint programme (CP-SAT) and as a MILP with big- M non-overlap (Gurobi), under identical budgets.

Instance	oracle	z	trucks	columns	CG iters	time (s)
I_4.10.2.90	cpsat	45 259	29	66	2	335
	gurobi	45 418	29	45	1	271
I_4.10.4.120	cpsat	51 967	36	80	2	341
	gurobi	51 133	35	74	3	288
I_4.5.2.60	cpsat	14 728	9	56	2	394
	gurobi	14 733	9	55	2	353
I_4.5.3.90	cpsat	21 724	14	86	4	232
	gurobi	21 934	14	84	7	225
I_4.5.4.120	cpsat	28 290	17	81	3	300
	gurobi	28 327	17	66	3	300
I_8.10.2.120	cpsat	62 711	43	99	2	517
	gurobi	62 850	43	82	2	426
I_8.10.3.60	cpsat	32 238	23	105	2	563
	gurobi	32 886	24	104	2	502
I_8.5.2.90	cpsat	21 177	14	117	3	300
	gurobi	21 400	14	111	5	262
I_8.5.4.60	cpsat	—	0	73	2	359
	gurobi	21 900	16	80	2	382
mean	cpsat		23.1	86	2.5	373
mean	gurobi		22.3	78	3.0	334

3.4 Discussion

A volumetric capacity is not a conservative approximation. Across the 36 instances SCPTOP dispatches 723 truck loads. A constraint programme proves that 503 of them — 70% — cannot be placed in a trailer at all: there is no assignment of their containers to stacks that respects the height budget and the crush limit and whose footprints fit on the floor without overlap. Only 17 loads are demonstrably loadable; the rest are undecided within the time limit. SCPTOP’s plans are on average 17% cheaper than MATH-3D-SCPTOP’s, and that entire margin is bought by loading trucks beyond what geometry allows. A planner handed such a plan discovers the shortfall on the loading dock.

The effect is strongly modulated by container size. At $v = 60$, where each RTI occupies about a sixtieth of the trailer, 93% of the volumetric loads are impossible and the apparent saving rises to 33%; at $v = 90$ it falls to 54% and 7.8%. Coarse containers leave large unusable remainders once footprints and layer heights are respected, and it is exactly that remainder a volumetric model spends. The direction of the error is worth stating plainly: a volumetric capacity is not a safe approximation that merely loses a little precision — it is systematically optimistic, and most so when the containers are large, which is the regime of interest in automotive procurement.

Feasibility by construction. Every one of the 36 plans MATH-3D-SCPTOP returns passes an independent truck-by-truck verification against (37)–(42), performed by code that shares nothing with the solver that produced it. That is not a coincidence of the search but a property of the formulation: a column is a witnessed layout, so a plan assembled from columns is executable by construction, and can be handed to a loading crew as a drawing. The realised utilisation confirms the loads are not merely legal but full: the least-filled truck in a plan reaches 68% of the binding resource on average, never below the $\alpha = 0.55$ floor.

Against the procedure in use. On the case study built from the industrial catalogue, MATH-3D-SCPTOP reduces total cost by 11.4–12.8% and the number of truck dispatches by 17–18% relative to the company’s average-mix procedure, consistently across the low-, normal- and high-variability scenarios. Against the stronger hand-drawn baseline — regular grids plus two-zone layouts, which is more than the planner actually maintains — the reduction is 10.5–11.9%. Two further observations matter more than the percentages. First, the advantage does not erode as demand variability grows; if anything it widens slightly, because a fixed set of average-mix configurations tracks a varying programme less and less well. Second, on three of the nine scenarios the average-mix procedure cannot serve the demand programme at all with its configurations, whereas the matheuristic finds a plan in every case. A procedure that occasionally has no feasible answer is a different kind of problem from one that is merely expensive, and it is invisible until the loading configurations are modelled explicitly.

Pricing technology. The subproblem was solved both as a constraint programme and as a MILP with the big- M non-overlap encoding (16)–(19), under identical budgets (Table 7). The two agree on value wherever both close the subproblem, which is a useful cross-check on the two formulations. Over the small and medium instances the constraint programme returns plans 0.4% better on average at comparable cost, and on individual pricing calls it is typically three to four times faster to prove optimality. We therefore use CP-SAT for the large instances. The margin is small enough that the choice is one of convenience rather than necessity — but it does confirm that the disjunctive structure of non-overlap is better served by a global constraint than by four big- M inequalities per pair of slots.

On bounds, and what the method does not deliver. The lower bounds are the weak point of these results and we report them as such. Certification requires the packing subproblem to be closed to proven optimality at the final pricing sweep, which succeeds on only 3 of the 36 instances; elsewhere the Lagrangian bound (49) applies, and it is loose — a mean gap of 28.6%, which reflects the bound rather than the plans. Where the compact model can be solved, the true gaps are far smaller (Table 6) — a median of about 2%, with two outliers on the two-product instances where the plan uses only a couple of trucks and the objective is dominated by inventory cost, so that a single over-delivered truckload moves it appreciably. That is the honest way to read the large-instance gaps: as an artefact of the bounding argument rather than a measure of the cost being left on the table. The runs behind Table 6 allocated four solver threads per instance, which starves the pricing sweeps; the outliers shrink substantially with more threads, and are reported as they stand. Closing them would require embedding the column generation in a branch-and-bound tree rather than the depth-first dive used here, which is the natural next step and is beyond the scope of this paper.

Where each method belongs. The compact model is not merely slower than the matheuristic on small instances — it is better, and often dramatically faster, solving instances with two or three products and two or three periods to proven optimality in seconds. The case for the decomposition is not that it dominates everywhere; it is that the compact model stops producing anything usable well before industrial sizes are reached, while the matheuristic returns verified plans on every instance in the benchmark within twenty minutes. The two are complements, and the crossover is where the placement block starts to dominate the search.

References

Moreno, Juan, Josefa Mula, and Raul Poler (2024). “Route Clustering Algorithms for Procurement Transport”. In: *Data Science and Applications*. Springer.